

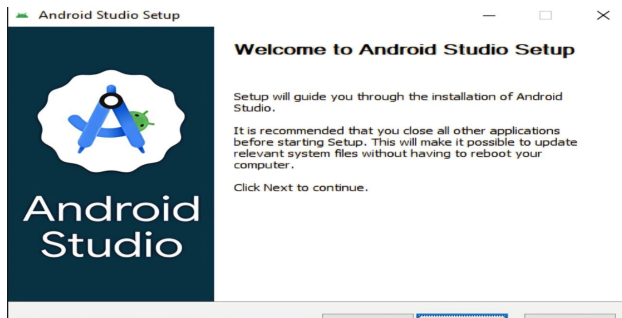
BIT/2024/57501

BIT4107:MOBILE APP DEVELOPMENT

WEEK1

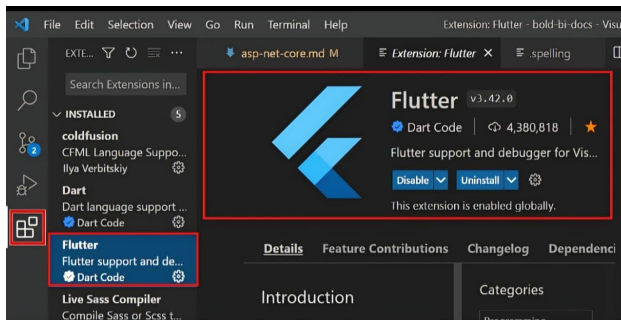
Development tools

.Android studio



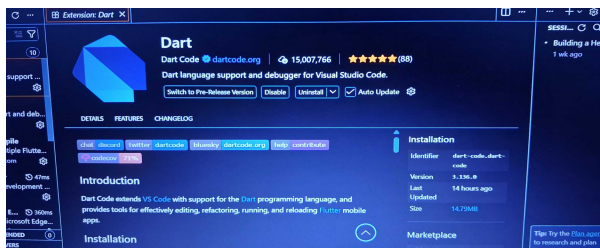
Used for writing code and managing files and debugging.

.Flutter extension.



Used to enable flutter support on the IDE.

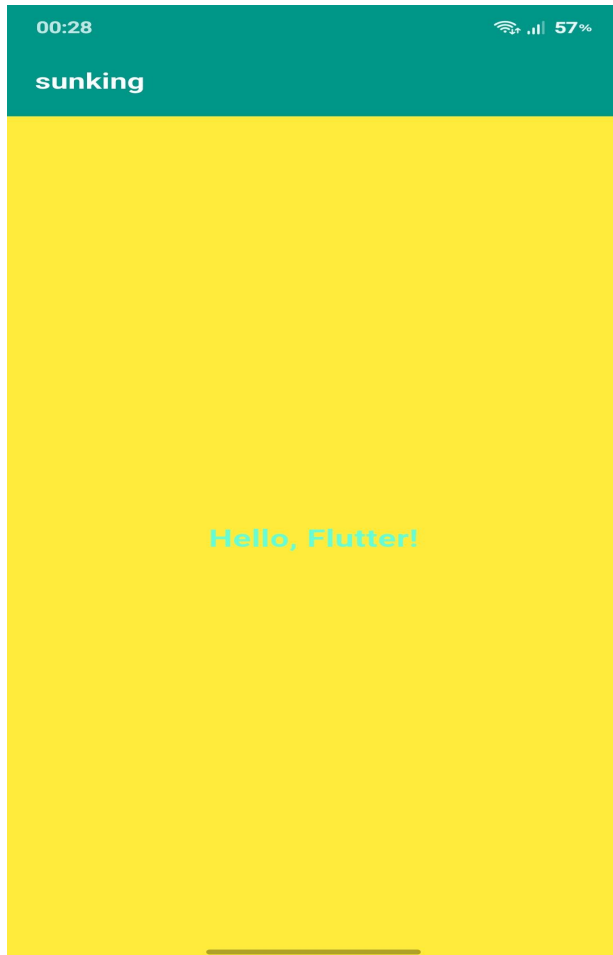
.Dart



Programming language used to write flutter applications.

.Android emulator

Virtual devices used for testing the app UI



Screenshot of app name changed,,background color and font size.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
```

```
const MyApp({super.key});

@override
Widget build(BuildContext context) {

  return MaterialApp(

    title: 'Custom Flutter App',

    debugShowCheckedModeBanner: false,

    theme: ThemeData(

      useMaterial3: true,

    ),

    home: const HomeScreen(),

  );
}

class HomeScreen extends StatelessWidget {

  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {

    return Scaffold(

      backgroundColor: Colors.blueGrey[900], // Background Color

      appBar: AppBar(

        title: const Text('My Custom Title'), // App Title

        backgroundColor: Colors.teal,

        foregroundColor: Colors.white,

      ),

      body: const Center(
```

```
child: Text(  
  'Hello, Flutter!',  
  style: TextStyle(  
    fontSize: 36.0, // Font Size  
    fontWeight: FontWeight.bold,  
    color: Colors.tealAccent,  
  ),  
,  
,  
,  
);  
}  
}
```

Source code of the created flutter app.

PERSONAL REFLECTION

Installing the tools was challenging at first due to errors and configuration issues, but it improved my problem-solving skills. I gained practical experience in setting up development environments and became more confident in using modern software tools for app development.

WEEK 2 ASSIGNMENT

Task management app

23:33 64%



Create Account

Join TaskMaster to stay organized

 Full Name

 Email Address

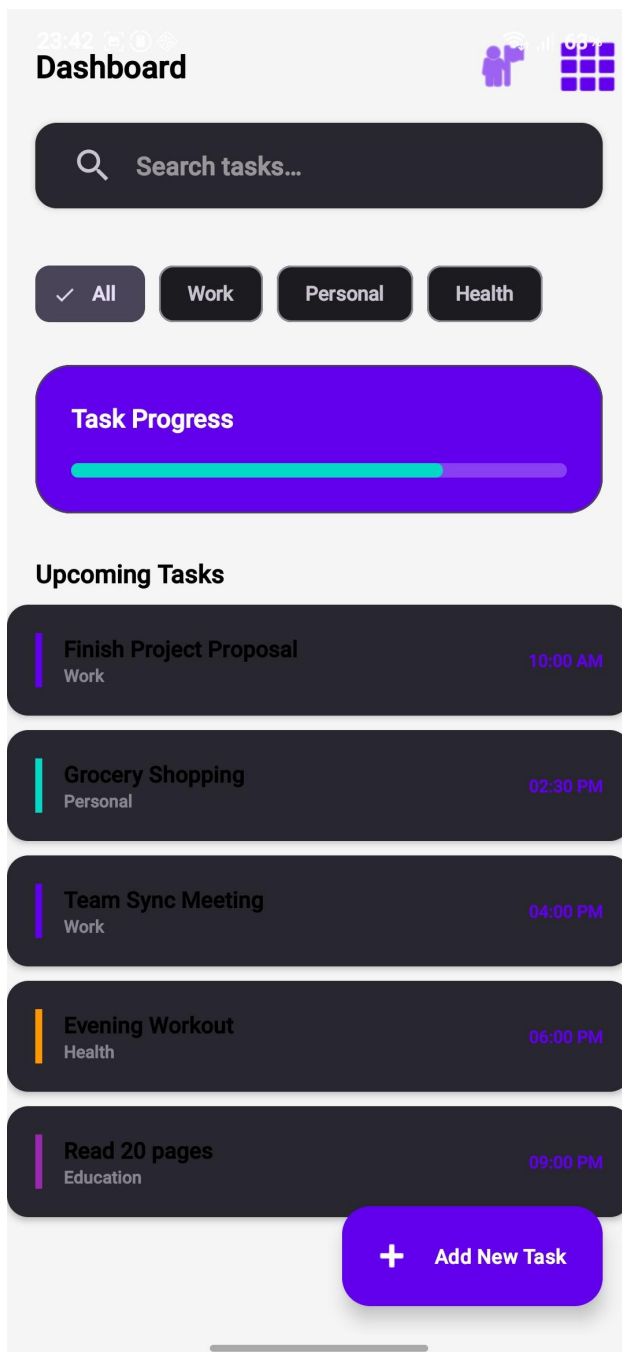
 Password



Sign Up

Already have an account? [Login](#)

Screenshots of the login page that helps you navigate through the app by keying in the email address and passwords so as to log in or sign up.

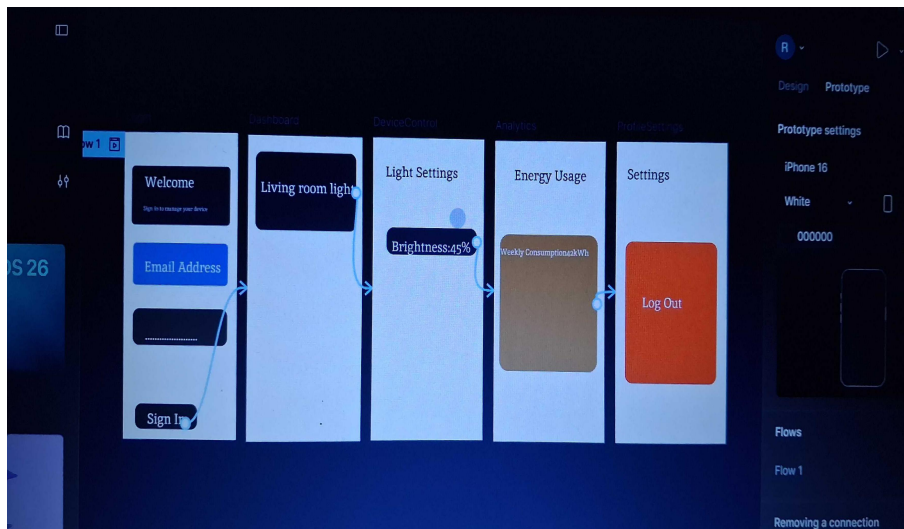
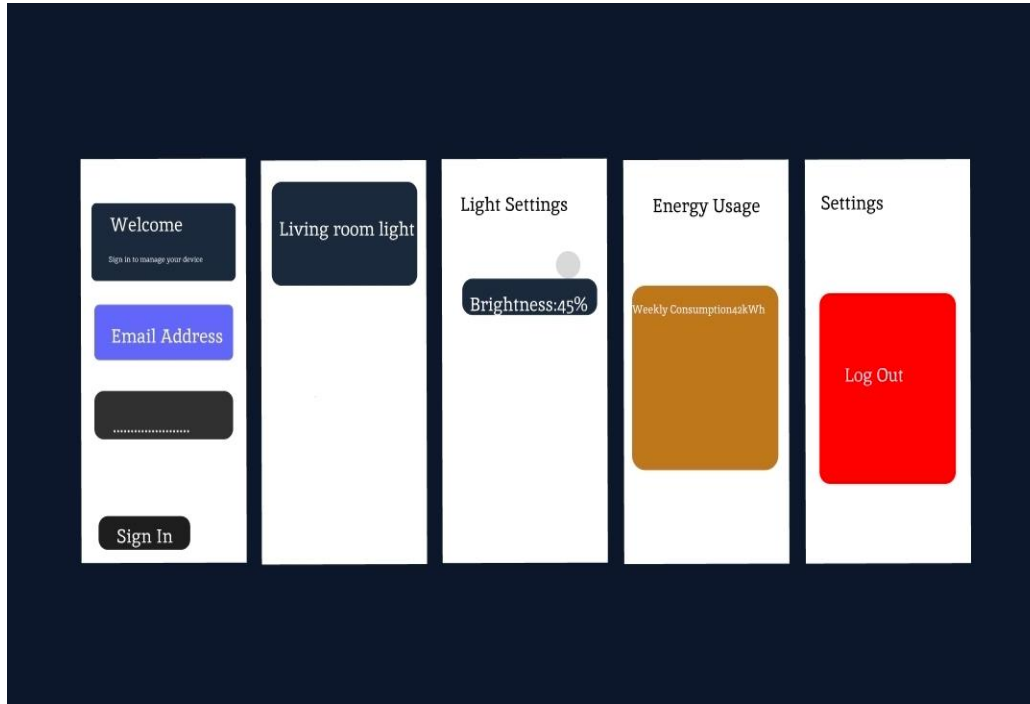


This is the dashboard that has features that helps you search and add new tasks and also helps you track and check you progress of the tasks.

WEEK3

UI PROTOTYPE

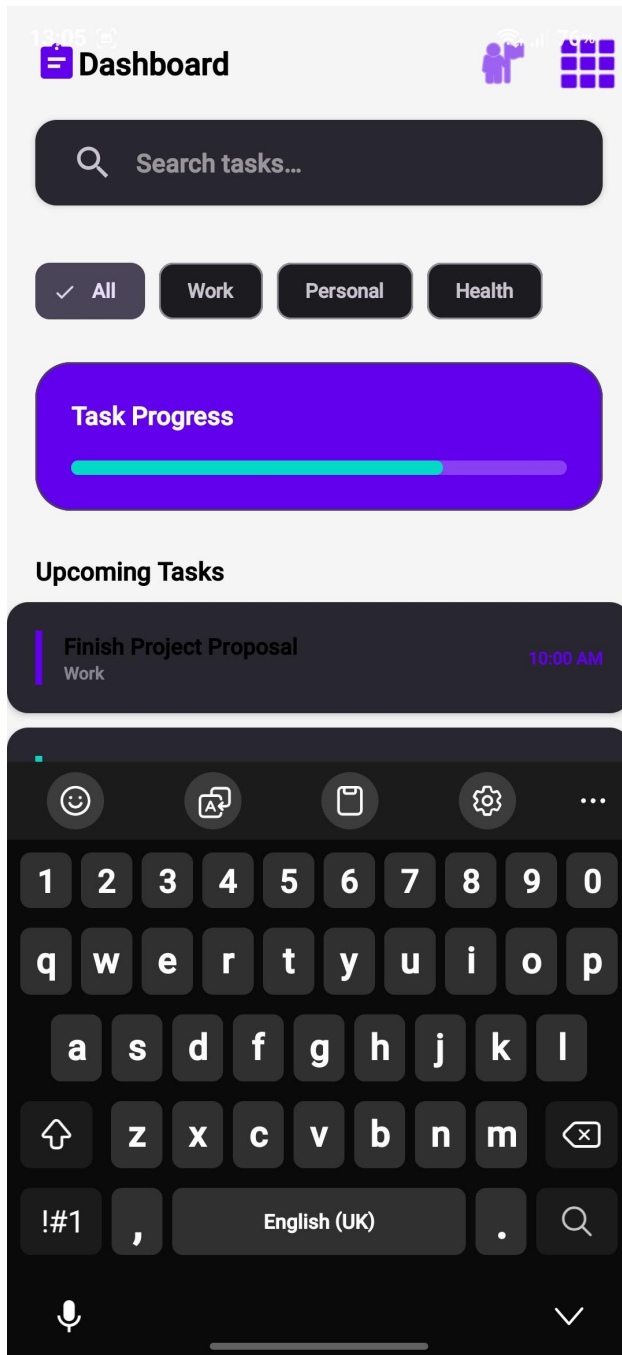
Tools used: Figma



Screenshot showing clear navigations from login page screen so settings screen.

WEEK 4

DATA MANAGEMENT



Screenshot of the add functionality

12:05



Create New Task

76%

Quick Suggestions

Meeting

Workout

Grocery

Study

Shopping

Reading

Relaxing

Cooking

Task Name

Description (Optional)

Category

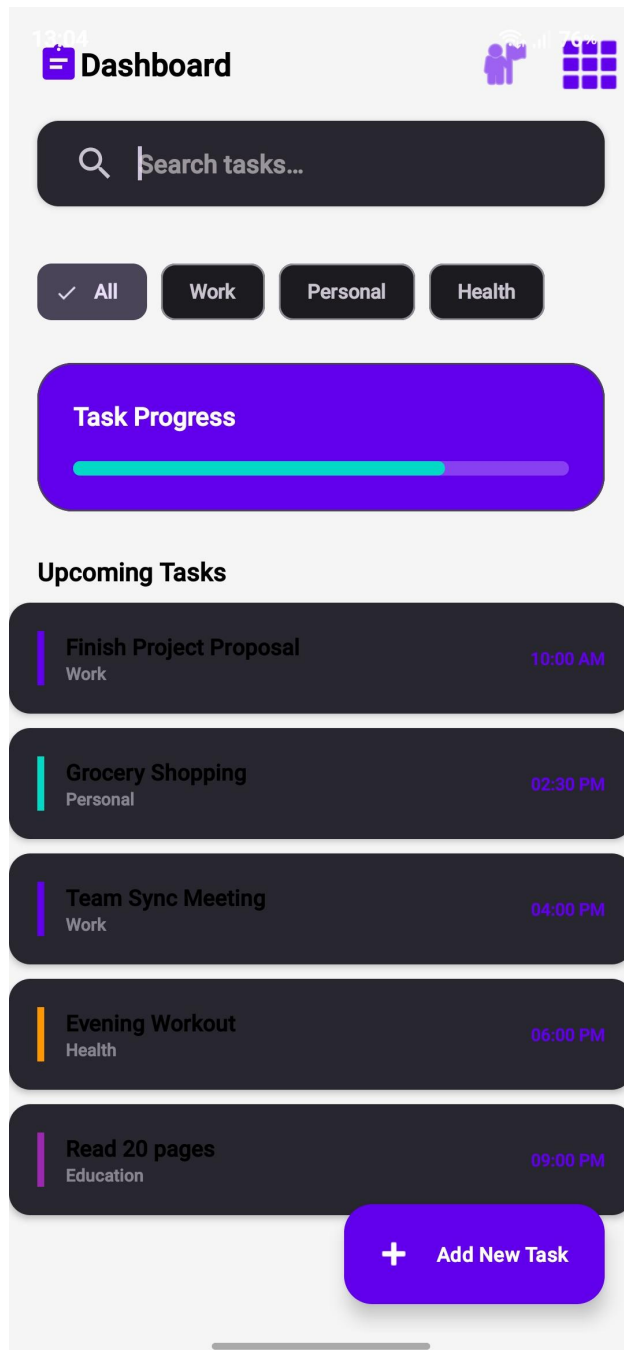


Select Date

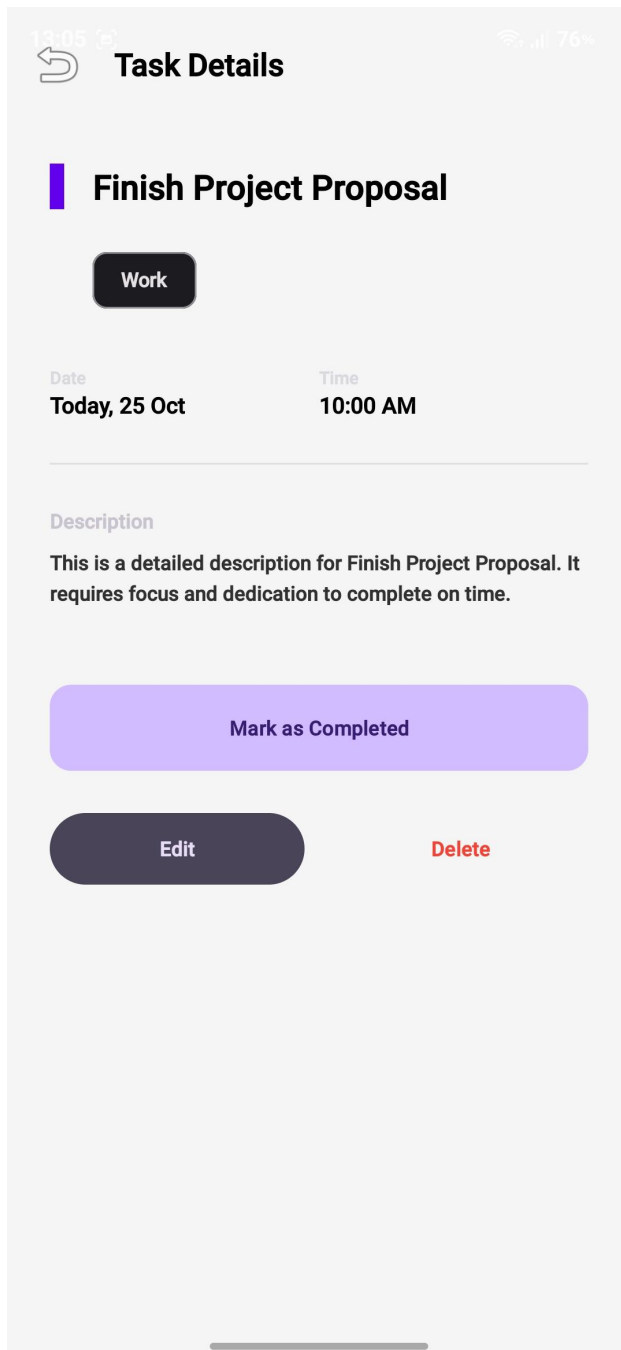


Select Time

Save Task



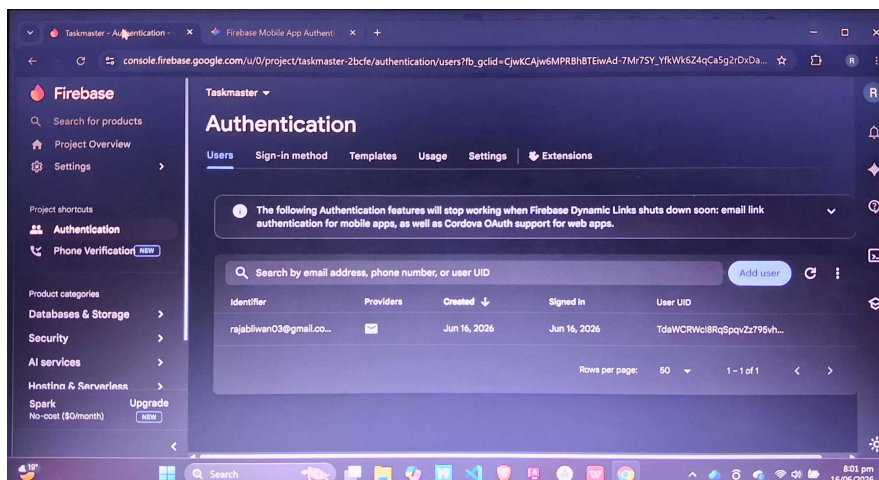
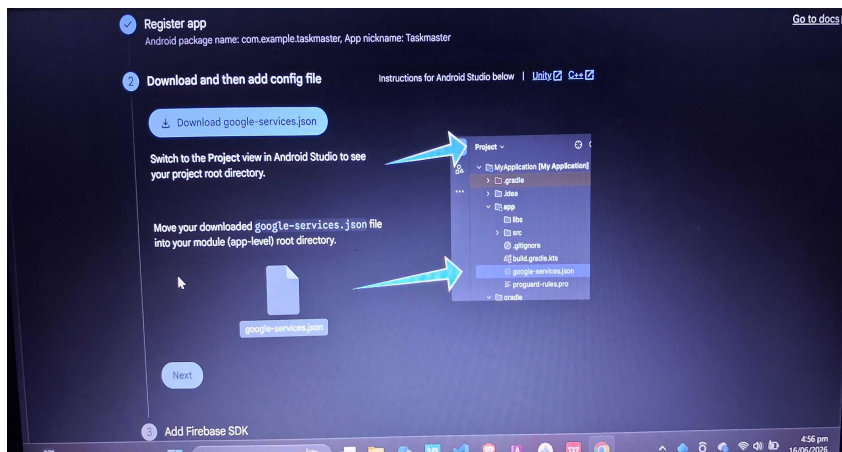
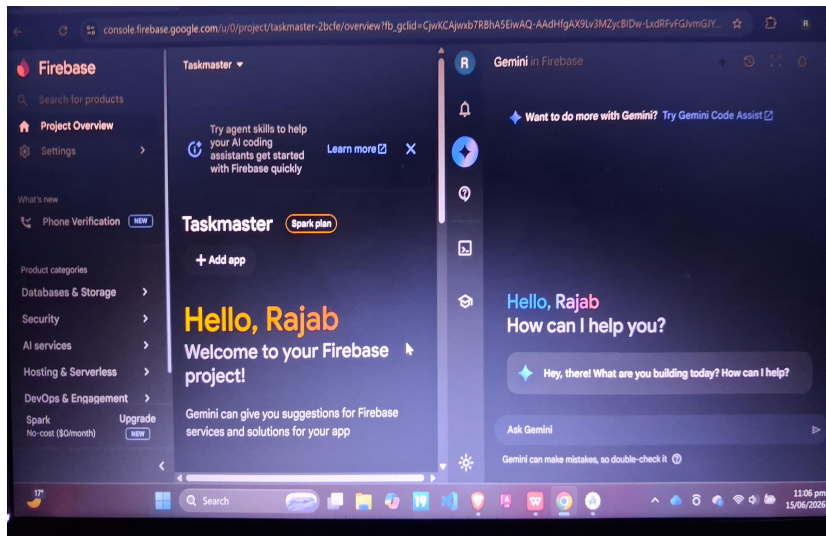
Add task functionality

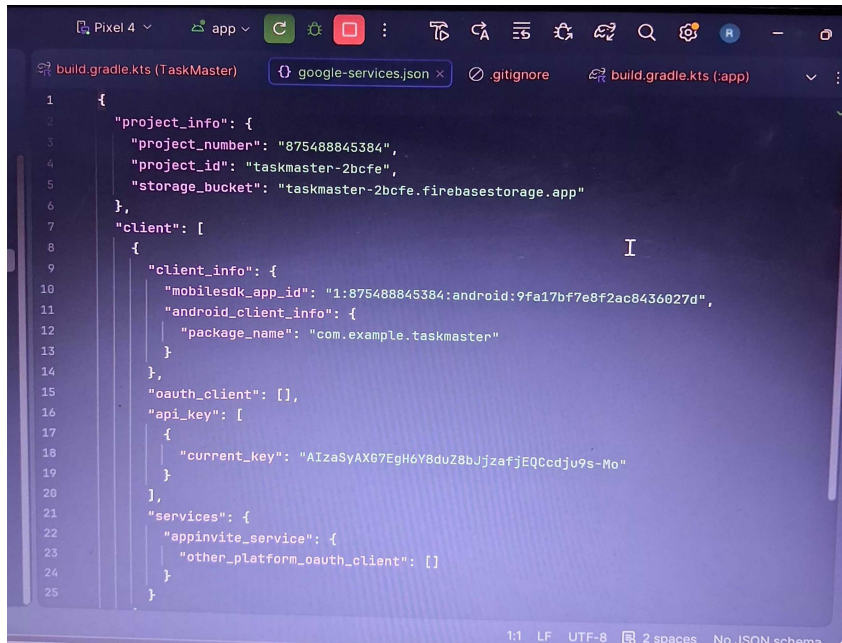


Delete function.

WEEK5/6

NETWORKING & APIs



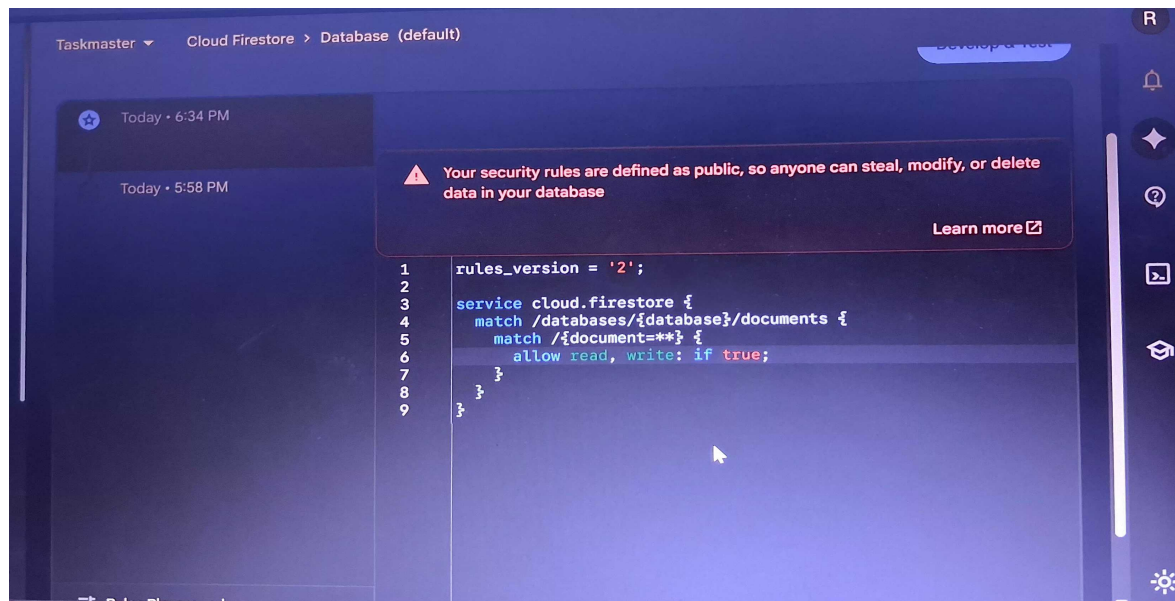


Screenshot of network connection string.

Personal reflection.

Implementing Firebase Auth with Jetpack Compose successfully validated my local UI login logic and state transitions on the emulator. However, debugging a silent cloud-sync mismatch taught me how to navigate frozen Logcat daemons, manage emulator caches, and understand how modern BaaS architectures replace traditional connection strings with JSON configurations.

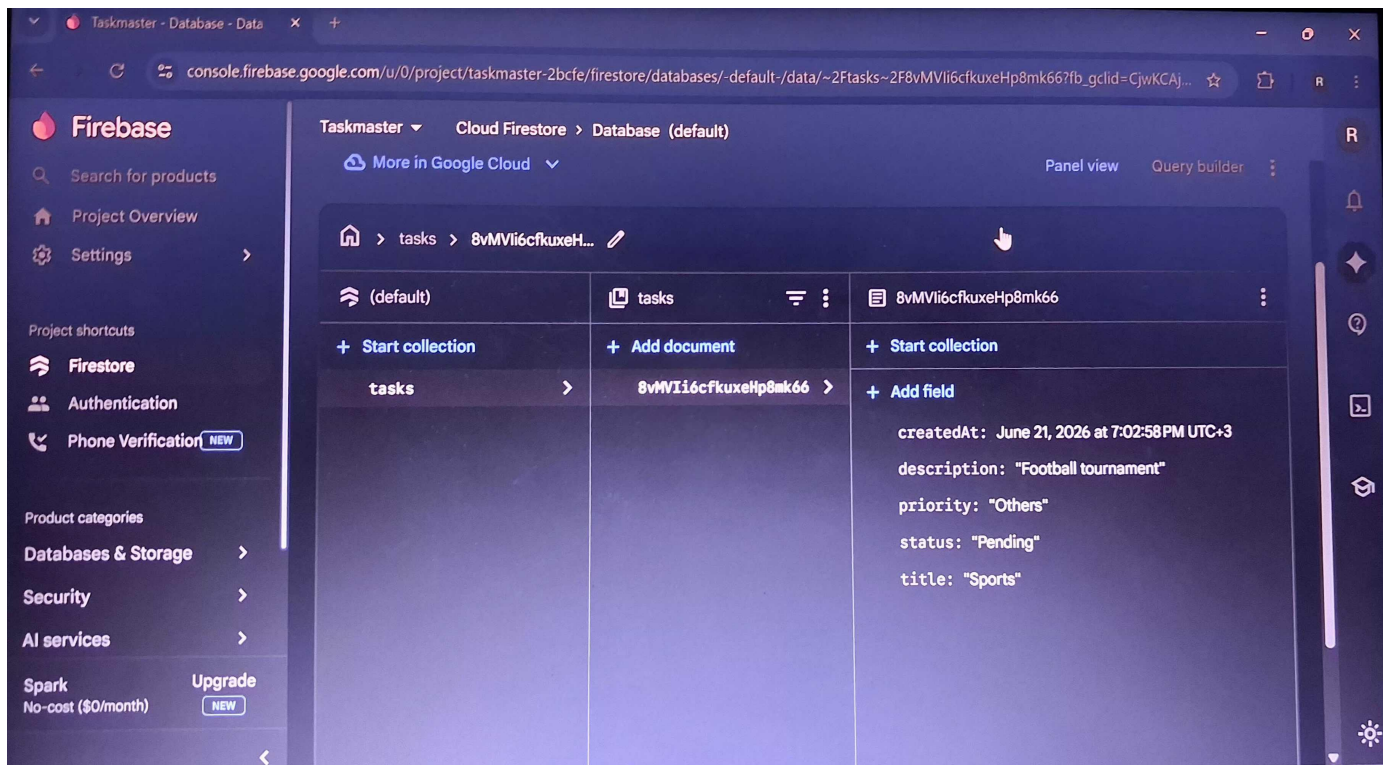
WEEK 7: DATA STORAGE AND DATABASE INTEGRATION IN MOBILE APPLICATIONS



Cloud Firestore security rules configured to permit incoming data writes during the development and testing phase.

Taskmaster user interface displaying a populated task creation form prior to database submission.

Taskmaster user interface displaying a populated task creation form prior to database submission.



Real-time Cloud Firestore Data tab demonstrating successful data persistence and collection generation.

Personal reflection: Integrating Cloud Firestore with the Taskmaster app provided invaluable, hands-on experience in bridging a native Android frontend with a live cloud database. Overcoming configuration hurdles like network permissions and security rules highlighted the critical role of systematic troubleshooting in software engineering. Seeing local input successfully populate the cloud console in real-time significantly boosted my confidence in developing functional, full-stack applications.

WEEK8:HANDLING USER INPUTS AND EVENTS

16:06

30%

Waiting for interaction...

Type something here...

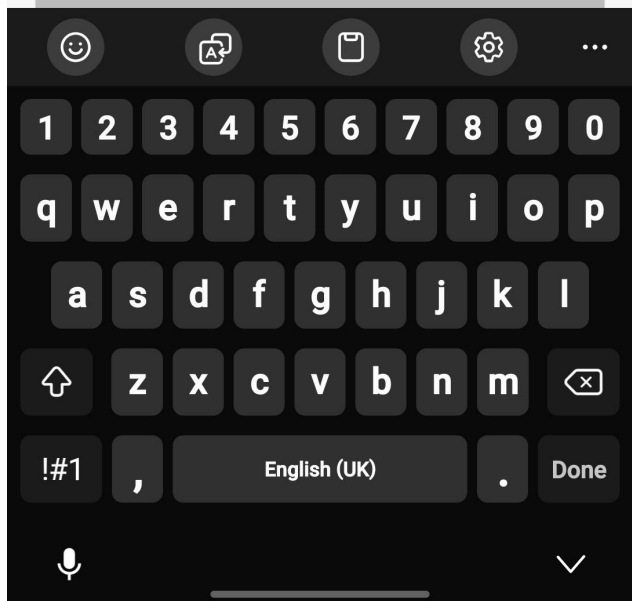
Touch Zone (Swipe/Tap here)

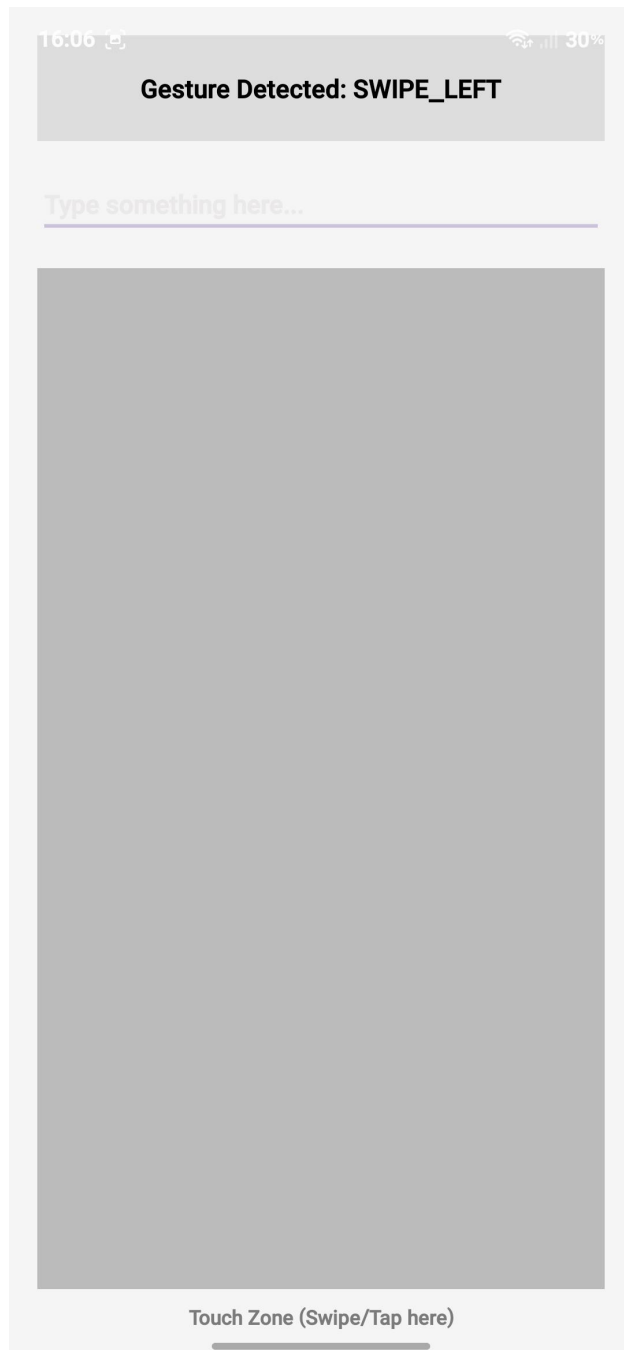
16:06

30%

User typed: Task management

Task management





Personal Reflection

Building this interaction framework shifted my perspective from writing flat code to designing highly scalable software architectures. Seeing abstract concepts like the Single Responsibility Principle seamlessly bridge user touches with system metrics was incredibly rewarding.

WEEK 9: DEVICE FEATURES INTEGRATION.

16:43 N [signal icons] 46%

Create New Task

Cooking

Task Name

Description (Optional)

Category **Personal** ▼

Select Date **Select Time**

Date: 03 Jul 2026
Time: 03:00 PM

Capture Photo & Location

Save Task


Location: -1.050635, 37.115726

Screenshot showing the task form with a captured image preview thumbnail visible and the GPS coordinates clearly displayed on the screen.

WEEK 10: COMPREHENSIVE APPLICATION DEVELOPMENT, INTEGRATION AND TESTING.

09:58 Create New Task 46%

Quick Suggestions

Meeting Workout Grocery Study

Shopping Reading Relaxing

Cooking

Task Name

Task name is required

Description (Optional)

Category

Health

Select Date Select Time

Capture Photo & Location

Save Task

Form input validation error,,a task title is require in order to save the task.

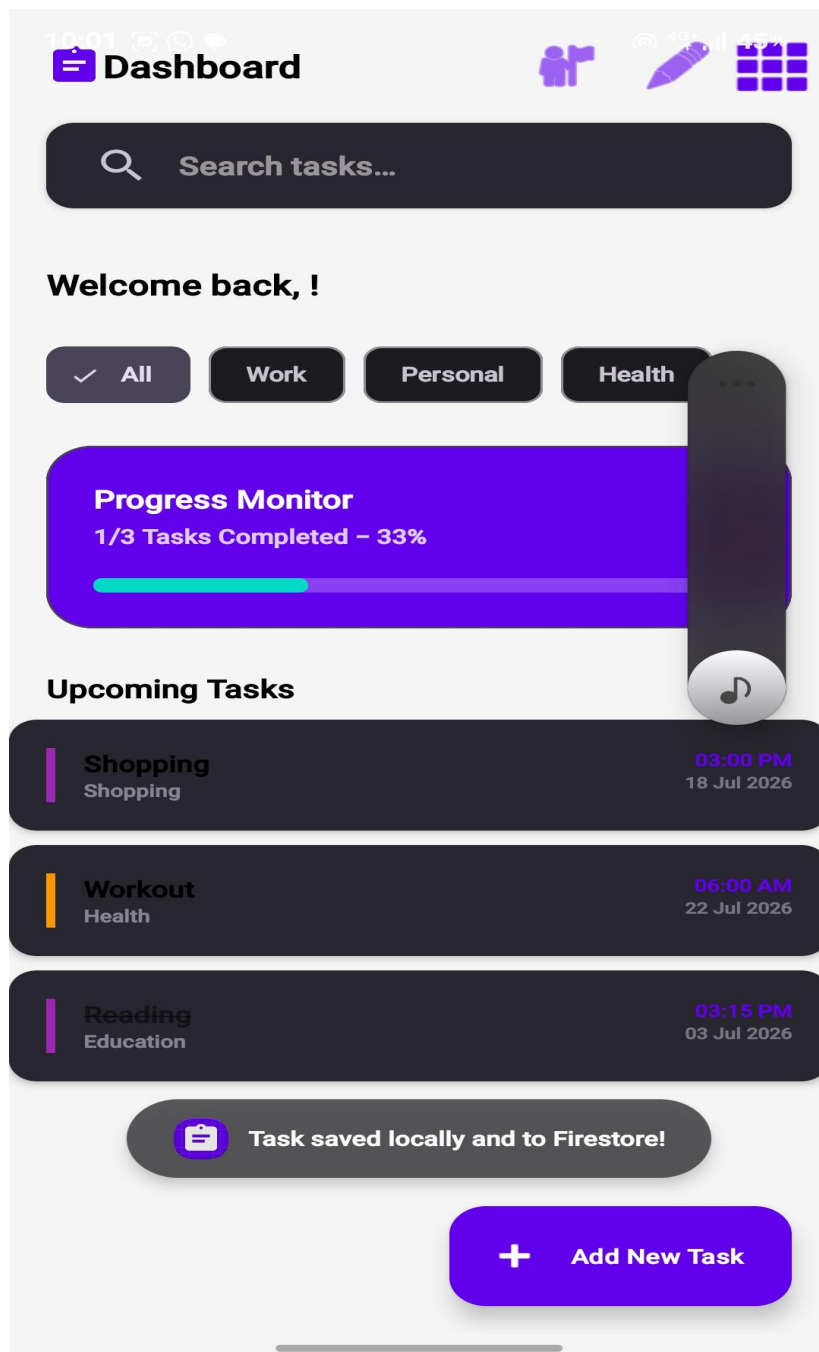
Bug reports:UI display regression

Description: Tasks saved successfully into the SQLite database file did not render dynamically inside the main dashboard RecyclerView list screen.

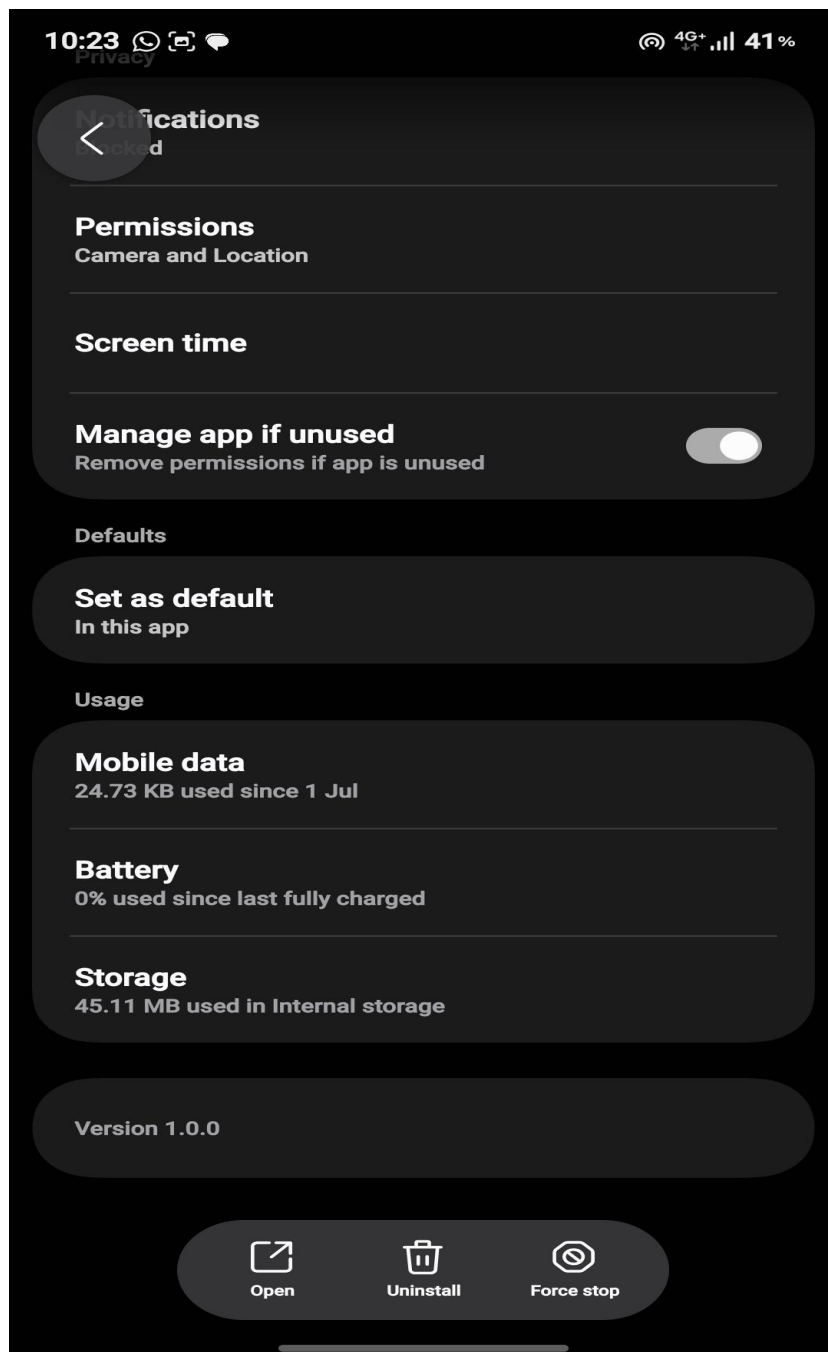
Root Cause: The database reading method was being invoked exclusively inside the onCreate() method lifecycle block. When returning from AddTaskActivity, the screen was re-displayed, but the data adapter list stayed completely static.

Resolution: Moved the database load functionality and the adapter data reset trigger call directly inside the onResume() Activity lifecycle callback block.

WEEK 11: APPLICATION DEPLOYMENT AND PUBLISHING



System success notification,, "task saved successfully"



Build configuration: version 1.0.0